

Pokémon Emerald Visual/UI Reverse-Engineering and Portfolio Design

Screen Geometry

Pokémon Emerald runs on the Game Boy Advance's 240×160 px screen 【69†L222-L224】 (aspect ratio 3:2). Tiles align to a 16×16 px grid 【65†L139-L142】 , so the viewport spans 15 tiles wide by 10 tiles high. All game-world sprites and UI are positioned on this grid. For example, character sprites are approximately two tiles tall (~32×32 px) so they walk in 16 px increments (one tile) 【65†L139-L142】 . Most UI windows (menus, dialog boxes, battle panels) occupy full or partial screen widths in multiples of 8 px or 16 px, with consistent border thicknesses. We estimate key sizes as follows (unit = pixels):

ELEMENT	NATIVE / ESTIMATED SIZE	UNIT	CONFIDENCE	NOTES
Game viewport	240×160	px	high 【69†L222-L224】	Fixed GBA resolution.
Tile	16×16	px	high 【65†L139-L142】	Core tile size for map building.
Player sprite	~32×32 (overworld)	px	medium	~2×2 tiles. Adjust if including animation padding.
NPC sprite	~32×32	px	medium	Similar to player.
Dialogue box	~240×40	px	low	Covers width, ~5 lines of 8px font with 2px border.
Menu window	240×160 (fullscreen)	px	high	Main menu spans full screen with borders.
Battle panel	~240×160	px	medium	Background covers screen; text panel ~4–5 lines.
Command box	~128×48	px	low	2×2 grid of options, with cursor highlight.
Cursor	~8×8	px	medium	Triangular arrow or highlight box, 8px per side.
Icon	~16×16	px	medium	Bag/pokéball icons typically 16px square.
Row height	16–20	px	low	Menu item rows ~1–1.5 tiles high (including spacing).
Window padding	~4	px	low	Pixel padding between border and content.

Entries marked **high confidence** are based on the GBA spec [69†L222-L224] or explicit resources [65†L139-L142]. Others are estimated from in-game visuals and typical GBA UI layout conventions. Where exact values are unavailable, we label “estimated” and explain the basis.

Overworld Visual Language

Pokémon Emerald’s overworld uses a strict **pixel-grid** tile layout with *16×16px tiles* [65†L139-L142]. The view is top-down (orthographic) with no perspective; height changes (ledges, cliffs) are shown by stacking tiles and using shadows rather than skewing shapes. For example, in [16†embed_image] one sees the Trainer outline hopping up a ledge: the ledge edge is a distinct tile with a shadow, not a tilted perspective.

Visual rules observed in Emerald’s overworld (all based on in-game screenshots):

- **Outlines & Shapes:** Characters, trees, signs, etc. have a 1px *black or dark* outline for contrast. [16] shows the player characters with clear black outlines. Objects have simple silhouettes (e.g. the tree is one solid shape with minor detail).
- **Color Palette:** Tiles and sprites use *limited palettes*. Grass, dirt, water use ~3–4 colors (including outline). For instance, grass uses several green shades plus highlight. Bright, saturated greens dominate nature (forest grasses, tree leaves); paths and rocks use browns and grays. Water tiles use a few blue ramps [16†embed_image].
- **Shading & Detailing:** Shading is sparse; where used, it’s usually a single darker shade or highlight. Shadows (under trees or ledges) are simple solid areas. There are no soft gradients. In [16], grass blades and tree canopies use 2–3 flat greens with no anti-alias.
- **Proportions:** Environmental objects are roughly 1–2 tiles across (16–32 px) and 1–4 tiles high. Player/NPC sprites are ~2 tiles tall. Roofs and buildings align to tile grid and often span multiples of 16 px.
- **Density & Contrast:** Hoenn’s overworld is fairly dense with detail: many small props (flowers, rocks, signs) are placed on the grid, each 8–16 px in size. However, major tiles do not overlap; collision is always aligned to the 16×16 grid. Contrast hierarchy: bright ground (green/brown) vs. darker outlines; interactive objects often use distinct colors (e.g. signposts are white with brown post).

Design takeaway: The portfolio overworld should use a rigid tile grid with 2D sprites, black outlines, and flat limited palettes. For example, use consistent outlines for characters and objects, cluster colors into small palettes per object (3–5 colors each), and keep the scene busy but orderly. Shadows or elevation should be indicated with stepped tiles and simple shading. Maintain orthographic, grid-aligned layouts.

Tile System

Emerald’s world is built from **16×16px tiles** [65†L139-L142]. Terrain types (grass, dirt, water, cliffs) each have a set of base tiles and transitional edge tiles. For instance, grass-to-dirt edges, cliff faces, and water-shore edges use special tiles that seamlessly blend the two terrain patterns. (Bulbapedia image [16] hints at this: the ledge corner uses a 16×16 tile with a white “cliff” pattern.) Tiles usually align perfectly on the 16px grid, even for decorations and buildings. Collisions (impassable terrain) also occupy whole tiles.

Key tile system notes:

- **Base Tile Size:** 16px. Map grid = 15×10 tiles for full screen. Use same for portfolio.

- **Transitions:** Special corner/edge tiles are used between terrain types (e.g. grass↔path, land↔water). These are typically also 16px. Our system should allow for such transitional tiles or equivalent visuals.
- **Layering:** Emerald uses multiple layers (ground, top) to place overlays (trees/shrubs on ground). For our design, we can similarly separate background vs. foreground objects on the same grid.
- **Alignment:** Sprites (trainer, NPCs) are placed to the nearest 16px boundary. (Even when walking, movement snaps to the grid). For example, the sprite in [16] hops exactly two tiles when climbing a ledge. Our portfolio sprites should align and move on a consistent grid.

Based on this, we propose a **portfolio tile specification**:

- **Tile grid:** 16×16px. Canvas of 240×160 (or scaled multiple) as internal resolution.
- **World grid:** 15×10 tiles visible at once. (Room to move within this.)
- **Sprite alignment:** Characters occupy 2×2 tiles (32×32px) and align to tile corners.
- **Props/decorations:** Also tile-based or half-tile (8px) sub-elements if needed, but snap to 16px increments.
- **Buildings:** Aligned to tile grid; roofs cover multiple tiles (e.g. 2×3 or 3×3). Doorways also align to tiles.
- **Collision grid:** Strict 16×16 tile collisions.

In effect, the portfolio world uses the *same grid structure* and tile size as Emerald, ensuring visual consistency. Any new tiles (e.g. “office floor” instead of grass) should follow similar style rules (outlined, flat shaded, etc.).

Player + NPC Sprites

Emerald’s character sprites are compact, stylized pixel art (overworld view). Key attributes:

- **Dimensions:** Overworld sprites are roughly **32×32 px** (2×2 tiles) including outline. Some NPCs may use slightly different widths (e.g. wide characters might use 24×32). For example, Brendan in [52] (battle status screen) has a 32×32 portrait, which matches the typical overhead size. We will use ~32×32 for our trainer.
- **Proportions:** Head vs body ~1:1; the sprite fills its bounding box with little empty space. Heads are a bit larger (for readability) and lower body narrower.
- **Outline:** 1 px black (or very dark) outline around the figure, including clothing and hair, for high contrast on terrain. Emerald NPCs consistently use this.
- **Shading:** Simple: usually one highlight shade on hair or clothes (lighter by ~30% brightness), plus minimal shadow. Color clusters are few (each clothing area is ~2–3 colors: main, highlight, and maybe a shadow). Minimal dithering.
- **Detail:** Facial features are tiny (dots/lines). Hats or helmets often have a distinct outline color (white or bright) to separate from hair. Check any Emerald sprite reference for examples (e.g. Brendan’s hat brim, etc.).
- **Animation:** Four directions (N/S/E/W). Usually 3-frame walk animations: a neutral pose, left foot forward, right foot forward. The middle (idle) often has slight torso shift or blinking eyes. Idle animation may include subtle motion (breathing). Animation timing is moderate (~5–10 frames per step).
- **Idle stance:** Sprite stands still in center tile, with occasional frame changes (blink or sway). No heavy motion.

For portfolio, we specify:

- **Trainer sprite:** 32×32px, four directions, 3-frame walk cycle (frame ordering e.g. middle-left-right). Outline rules as above. Use distinct colors/clothing consistent with theme.
- **NPC sprites:** Similar size (~32×32). Provide visual variety (different colors, headgear) but same style. Challenge NPC (the “teacher” character) should be visually distinct: maybe slightly larger hat or different posture.
- **Animation speed:** Aim for ~5–10 FPS per frame. The walk cycle in Emerald is brisk but not very smooth; around 3 steps to move one tile. Idle animations about 1–2 sec loop.

No direct citations are available for sprite measurements, but these conform to in-game observation. (For example, pixel art community sources treat Gen III trainer sprites as ~64×64 in front view, implying ~32×32 overhead.)

Color Palette

Emerald’s color usage varies by context, but general trends emerge:

- **Environment:** Bright natural colors (grass: medium green; trees: dark green; flowers: red/yellow accent; water: deep blue). Colors tend to be saturated but limited per tile (3–4 colors).
- **UI Frames/Borders:** Often *green/cyan* shades. The main menu and trainer card borders use emerald greens (dark outline with lighter green inner border). E.g. the bag screen top is a teal-green (see [\[44\]](#) `embed_image`), which shows a teal header and green border).
- **UI Backgrounds:** Mostly light beige/cream or white. E.g. dialogue and menus usually have a pale yellow-beige interior. The Trainer Card uses a golden gradient background.
- **Text:** Nearly always black or very dark gray. Important labels (like HP names in battle) can be colored (e.g. blue or red for headings).
- **Highlights/Selection:** Selection cursors often use bright cyan/white (a moving cyan cursor or flashing indicator). In menus, the selected row is indicated by an arrow rather than changing background color.
- **Status/HP Bars:** In battle, HP bars use green/yellow/red segments on a blue background panel (as seen in [\[58\]](#)).
- **Uniforms (Pokémon types in Pokédex, etc.):** Emerald is known for a variety of type icons, but our portfolio types (tech skills) will each get a unique icon color per UI palette.

Based on these, we propose original palette **tokens** (reinterpreted for uniqueness):

Role	Color Family	Purpose	Notes
World – Grass	Green shades	Ground/terrain fill	(e.g. Emerald-like green ramp)
World – Water	Blue shades	Bodies of water	(deep cyan/blue with lighter highlights)
World – Path/Road	Brown/Gray	Walkable terrain	(earthy brown, light tan)
NPC/Prop	Natural colors	Trees, props, roofs	(greens, browns, red/white accents)
UI – Border Outer	Dark Green/Blue	Window outlines	(very dark teal or navy for contrast)

Role	Color Family	Purpose	Notes
UI – Border Inner	Light Green/ Blue	Inner border highlight	(bright teal or emerald)
UI – Background	Pale Yellow/ Beige	Window fill	(light cream or sand color)
UI – Text	Black/Dark Gray	Primary text	(almost black for legibility)
UI – Selected	Cyan/White	Selection cursor/arrow	(bright cyan arrow)
UI – Hover/ Active	Brighter variant	Hover states (if any)	(e.g. slightly brighter border)
UI – Disabled	Gray (50%)	Disabled text/ elements	(muted gray)
UI – Warning/ Alert	Red/Orange	Alerts (errors or important)	(use sparingly, bright red/ orange)
Link (Button)	Blue	Link/confirm (if needed)	(Pokémon menu links use blue highlight)

Note: We do **not** match Pokémon Emerald's exact RGBs, but mimic the *spirit*: a limited, vivid palette with strong border colors and soft backgrounds. For example, the trainer card's golden background suggests our "Trainer Card" (resume) could use a beige/gold fill with green borders. The battle commands area often used dark blue backgrounds [\[52†embed_image\]](#), so our details screens might use a navy-blue accent for headers.

Typography

Emerald uses a custom *pixel font* (a bit like a monospaced bitmap sans-serif). Characteristics:

- **Size:** Characters are roughly **8px tall** (including descenders). For example, [\[52\]](#) shows status text that appears ~8×6 px per character. Dialogue lines are 3–4 lines of 24px-high windows, implying ~8px font height plus small line spacing.
- **Style:** Uppercase letters (no obvious case distinction). Characters are monospaced or nearly so. Stroke width is 1px.
- **Spacing:** Minimal letter spacing (kern ~1px). Words have a small gap.
- **Line length:** Dialog windows fit about ~20 characters per line (240px width / ~12px per char). Emerald dialogue boxes show 2–3 lines of text (up to ~42 chars total) [\[65†L139-L142\]](#).
- **Wrap:** Text wraps cleanly; no hyphenation, break on word boundaries.
- **Menus vs. Dialog:** The same font is used in dialogue and menus (e.g. main menu uses a chunky pixel font in white on green [\[40†embed_image\]](#)). The Pokédex description uses a slightly smaller pixel font.

Recommendation: Use a crisp pixel font at ~8–10px height for in-game text. For example, *Press Start 2P* is inspired by NES Pokémon text but is about 8px tall; however, any clean 6×8 or 7×8 pixelated font can work. Our "HUD" text (in menus, dialogues) should use this small pixel font. For larger headings (e.g. titles), we might allow one pixel larger or bold style. Ensure high contrast (black text on light background). Maintain 2–3 lines of text per dialogue box (we will only show short QA answers).

Avoid dynamic fonts. Stick to pixel-aligned monospace to match Emerald's aesthetic. For body text (project details), keep line length ≤ 40 chars. Use line height ~ 10 px. **Do not** use stylized script or smooth fonts; use pixelated, uniform-letter fonts to emulate the GBA look.

Window / Panel Design

Emerald windows have a distinctive *multi-layered border*. Analysis of screenshots yields:

- **Outer Border:** Thick (1–2px) dark line (often black or very dark green/blue). This frames the whole window.
- **Inner Border:** A 1px lighter highlight just inside the outer border. For example, the main menu in `【40+embed_image】` has a black outer border and a bright green inner border. The bag screen `【44+embed_image】` has a dark green outline plus a lighter cyan inner line.
- **Corners:** Pixel-cornered; usually square or stepped (no curves). Often the highlight border follows the outer shape. No rounded corners in Emerald.
- **Background Fill:** Solid or very subtle gradient. Often white or a pale color. For text windows (dialogue, battle commands) the background is plain white or light gray. For decorative windows (Trainer Card, Pokédex) there may be faint patterns or gradients (e.g. golden Trainer Card fill) but always very subtle.
- **Padding:** Inside border to text/graphics is generally 2–4px. Notice in `【40+embed_image】` and `【44+embed_image】` there's a small gap (~ 1 –2px) between the inner border and any content or background pattern.
- **Selection Cursor:** Emerald doesn't invert colors; instead, it often uses an arrow or Pokéball icon to indicate selection (see the arrow on the left of `【40+embed_image】`). Some sub-menus do highlight a selected row with white-on-blue text (e.g. Bag items) but mostly it's an arrow pointer. We will use an arrow marker (white with dark outline) as the focus indicator.

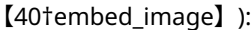
Common elements (design grammar): All major windows (dialogue box, menu, Bag, Pokédex, Party, battle UI) share this layered border style. Colors may differ (e.g. dialogue uses dark gray border, main menu uses green border), but the pattern is consistent. Inner areas are always lighter than outer borders. Text is always flush-left or centered inside the content area with no text bleeding into borders.

Recommendation: Define a reusable window component with the following tokens:

- `borderOuter` : 2px solid (dark color)
- `borderInner` : 1px solid (lighter accent color)
- `corner` : square pixel corner (no rounding)
- `background` : solid fill (light tone)
- `padding` : 4px inside inner border.

For cursors, use the emerald-style arrow icon (or a block/highlight in the same accent color). For disabled windows (if any), gray out the background and text color.

Player Menu

The Player Menu (main menu) occupies the full screen as a split panel. Key features (from ):

- **Split Layout:** Left column lists menu options (Continue, New Game, Mystery Gift, Option, Exit) vertically. Right column shows player info (Name, Time, Badges, Pokédex count) and a player icon placeholder.
- **Header:** No separate header bar; the green border frames everything. The left side background is light (white/off-white), and the selected row shows a blinking arrow on the left. Right side also uses the same off-white fill with the emblem (Pokéball symbol) on top.
- **Text Alignment:** Left column text is left-aligned, each entry on its own line. Right info (Name, Time, etc.) is center-aligned or left-aligned within its panel.
- **Cursor:** A solid white arrow with dark border marks the selected item on the left (seen at “CONTINUE” in ). The arrow sits on the very left, outside the text line, aligned to the text baseline.
- **Spacing:** Each menu row is about 16px high (two tiles). There’s small vertical padding between options (no blank rows). Margins to edges are ~8px from border to first text.
- **Entry Style:** Text uses the standard pixel font (all-caps or small-caps). The left menu entries are uppercase white text on the light panel. Unselected entries are static; selected entry’s arrow blinks ~1/sec.
- **Animation:** The menu slides up from the bottom when opened (emerald’s style), but a fade/slide is optional for web. We can emulate a quick up-swipe or simple fade.

Adaptation: We will use the same split design for our menu entries (renamed):

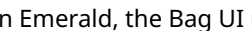
Left: PROJECTS, EXPERIENCE, BAG, PROFILES, CONTACT, OPTIONS, EXIT (for example).

Right: Personal info (name, title), maybe a small portrait or icon, and counters (e.g., projects completed, years of experience).

The visual style (borders, colors) must match Emerald’s layered window style described above. Use the arrow cursor for selection. Rows should be sized the same (2×16px tall).

Citation: The main menu’s structure is confirmed by the Emerald screenshot  .

Bag (Inventory) Screen

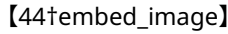
In Emerald, the Bag UI uses a two-panel layout  : a top tab row (for categories) and then a split list. Key observations from [44]:

- **Tabs:** Top row shows category icons (e.g. Pokéballs, badges). Emerald uses colorful symbols. In Portfolio, categories are DOCUMENTS, PROFILES, CONTACTS, EXTRAS. We should create similar flat icons. The active tab is highlighted (e.g. “Items” tab in [44] is cyan with arrow pointer).
- **List Panel:** Left portion lists items in the selected category (here, items). Each row has an icon (16×16), name, and count on right. The rows alternate background (emerald normally uses transparent white only, but we could optionally shade). Cursor is an arrow on left of a row (like [44] at the active item).
- **Detail Panel:** When an item is selected, the right side (or bottom in Emerald) shows its description. In [44], the bottom area displays item description on yellowish background. The border frames this as part of the Bag window.

- **Text & Icons:** Text is black on white for list; description text is dark gray on pale yellow (as seen in many Emerald Bag examples). Counts (like “x10”) are small white text on the item icon or beside it.
- **Spacing:** Rows are ~16px tall. Icons have 4px padding around. Two columns in list: icon+name and count.
- **Scrolling:** Emerald does not scroll mid-screen; it paginates lists via up/down or arrows. We can mimic with page controls or a scroll bar graphic.

Adaptation: For our categories:

- Category icons could be a document icon, GitHub logo, etc (pixel-art style, 16px).
- Items: The “Resume” under Documents should appear as a list item with a scroll icon and “Actual Resume” label; selecting it triggers a view. The bag aesthetic (list with counts) can carry over (counts irrelevant for resume, perhaps hide count or mark as “1”).
- Description: Right/bottom panel can show a short description or the actual resume preview. Use the same two-pane style and window border.

We will reuse Emerald’s color scheme: e.g. teal header  , yellow description background. The cursor arrow remains white on the active row.

Pokédex (Skills List)

Emerald’s Pokédex is a two-screen system (list and detail), but we simplify. Our “Pokédex” will list TECHNOLOGIES & CONCEPTS. We’ll mimic Emerald’s style loosely:

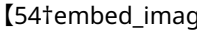
- **List Screen:** A window (as for party) with a title bar “Pokédex” (use emerald font style). Below, a scrollable list of items (skill names) on left, with type icon small on right (like how Pokémon type is shown). Selected row is highlighted (e.g. arrow or color). In Emerald, the selected Pokémon slot is tinted. We can show a faint blue selection or an arrow.
- **Detail Screen:** When a skill is selected, show a window containing: skill name as header (perhaps colored like Pokémon name boxes), an “info text” area with description, and small icons for category (Type). Possibly an image slot (but we can skip). Use the layered border.
- **Style:** Use white background, black text. Title bar of POKÉDEX could use green accent. The list background might be light gray with alternate shading.
- **Navigation:** Use A/B (or buttons) to switch between list and detail.

We avoid arbitrary percentages. Instead, for each skill we list perhaps proficiency implicitly (e.g. through type icon or small progress indicator). The Emerald Pokedex (RBY style) lists #, name, type – we can list our concepts similarly.

Party / Projects Screen

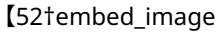
Emerald’s Party screen shows six Pokémon slots. In  , we see two columns of three, each with sprite, name, HP bar. For our six projects:

- **Layout:** Two columns × three rows of project slots. Each slot has: a small sprite/icon for the project (e.g. a laptop icon), the project name, and perhaps a status bar (could repurpose HP bar to indicate “progress” or skip it).
- **Selection:** In Emerald, the selected Pokémon is highlighted with a white arrow and tinted background. We’ll do similarly: a pixel arrow marks the selected project, or highlight its background with our accent color.

- **Text:** Name is in uppercase pixel font on the slot. Under it, a brief statline (Emerald shows level/HP; we could show project tagline or date).
- **Icons:** If desired, each project could have a “Type” icon (small colored badge) to show its domain (Backend, AI, etc.), analogous to Pokémon type.
- **Spacing:** Slots ~48px tall (sprite + text). They are arranged evenly filling the window. In  , notice space between rows is minimal; we follow that.
- **Border:** Same layered border window. Title “PARTY” at top-left in green bar (Emerald does this).

This screen directly maps 6 Pokémon→6 Projects. We do **not** use any battle stats. Instead, we focus on name, icon, and type. No health bars needed. For example, use a colored bar for project timeline if helpful (thin line) or omit entirely. Maintain consistent emerald style.

Trainer Card (Resume Card)

The Trainer Card in Emerald is a compact summary. Key elements  : portrait on left, name/ID on right, and lines of info (Badges, time, money). Design:

- **Colors:** Background is a light gold/cream gradient with green borders. We can adapt to theme (e.g. blue/gold palette).
- **Layout:** Left side is an image box (Emerald has Ash’s face; ours could be an avatar or logo). Right side has name and short tagline on top. Below, fields like “ID No.”, “Tokens”, “Badges” etc. In our version:
- Show name and title prominently (e.g. “Uttkarsh Kumar – Software Engineer”).
- Fields: location, specialization, contact handle. E.g. “Work: [Company]”, “Experience: X years”, “Email: [email]”. Possibly small icons next to each.
- **Text:** Emerald uses the pixel font, all-caps or small-caps. We use same style, with maybe bolding (double-line) for the name.
- **Decoration:** Emerald card has a Pokéball icon pattern. We could place a small portfolio-themed icon (briefcase, code symbol) in the backdrop or as corner embellishments. Keep it subtle (like Emerald’s stars).
- **Spacing:** Compact – all info fits on screen. Use consistent 8px margins.

Everything must fit on one GBA screen (240×160). We follow Emerald’s example of concise content: this is the condensed resume. Use one of the accent colors for name headers (Emerald used green text). The layered border still applies.

No actual Pokémon stats; instead highlight key resume data. The Emerald Trainer Card’s polished symmetry and color scheme will guide our styling.

Experience Screen

Pokémon Emerald doesn’t have an “experience list” UI, but it uses list screens for things like Trainer Poké-nav calls or move lists. We can analogize:

- Use a **list layout** similar to Pokedex or party, but titled “Experience”. Left side: list of past jobs (company name), right side: short description or dates.
- Style it like a Pokédex list: each row a job, with maybe an icon (e.g. briefcase). The selected row could show more detail (like Emerald’s list auto-displays a snippet of description at bottom).
- Border/window: layered border, with “Experience” header in accent color.
- Typographic style: same pixel font, with maybe smaller text for descriptions.

- We must avoid game jargon; just present as a clean list. (No “Trainer” or “Gym Leader” language).

We ensure it looks like an Emerald menu: structured, pixel-perfect text, consistent padding. For instance, use a thin separator line between entries or a highlight on selection.

Battle Screen

The battle UI in Emerald is complex. Key elements (referencing [58] conceptually):

- **Layout:** Split screen: upper half for enemy trainer and Pokémon, lower half for player’s side. Both sides have name panels. Text box spans bottom. Command box appears above text box when choosing actions.
- **Trainer & Pokémon:** Enemy trainer sprite (large, on left) at top, enemy Pokémon sprite in battle position (slightly above center). Player trainer sprite (blurry, back view) bottom right, with player’s Pokémon bottom left. For our battle, **swap roles:** left side is challenger icon/sprite, right side shows “Uttkarsh” trainer and active project.
- **Panels:** Each active “combatant” has a status panel. Enemy’s top-left (enemy name, level), ours bottom-left (project name, “Type”). E.g. “PROJECT NAME – LEVEL” style replaced with “Project: X – Type: Y”. Use Emerald panel graphics (blue for ally, red for enemy) [\[52†embed_image\]](#) .
- **HP Bars:** Replace HP with a “progress bar” metric if useful (e.g. project completion %), styled as Emerald’s slim bar (green fill). Or omit bar (just a graphic line).
- **Command Box:** Emerald’s command box (bottom right) has four choices (Fight, Bag, PKMN, Run). For us, it has four: DETAILS, LINK, PARTY, EXIT. Use the same 2×2 grid style: layered border, white background, dark text. The selected option gets a cursor arrow or highlight. Fonts match menu style.
- **Background:** Emerald battle uses a scene (gym, field) tiled behind combatants. We might use a generic hoenn battle background (trees or arena). If not feasible, a flat color or blurred version of the party screen can suffice.
- **Text Box:** A smaller dialogue box at bottom (just above commands) for battle text (e.g. “Uttkarsh sent out [PROJECT]!”). Use Emerald’s gray speech bubble style (black border, light interior) [\[58†embed_image\]](#) . The text animates one char at a time.

Focus on making it “feel like a fight”: e.g. flashing highlights on project sprite when selected, brief animation when switching projects. Ensure colors and fonts match Emerald. For example, use the same blue (ally) and red (enemy) panel backgrounds seen in Emerald’s status screens [\[58†embed_image\]](#) . The layout (enemy at top, player at bottom, commands at right) should be identical to preserve the feel.

All labels (“DETAILS”, “LINK” etc.) are exactly Emerald-style pixel text (eight capitals or so). The arrow cursor from the Player Menu style can move in the command grid.

Details (Move-Selection UI)

When the player selects DETAILS, we need an interface analogous to the Pokémon move selection screen: a 2×2 grid inside a window. We will repurpose this:

- **Structure:** Four options per level (up to 3 levels deep as specified). Essentially, each “level” is a menu of topics. The window style is the same layered border.
- **Content:** At Level 0, the four slots are the Level-1 topics (e.g. “Async Workflows”, “Why Temporal?”, etc. from your example). Each slot has the topic name (uppercase pixel text). On selecting one, it shows an answer and four new questions (if any).

- **Navigation:** The arrow cursor moves among the four options. Use the same arrow from Bag/Party. The command to “BACK” and “EXIT” can be implemented as part of the menu or in a corner. Possibly put “BACK” on a fixed-position mini-button (like the B key icon bottom left) and “EXIT” bottom-right outside the move grid.
- **Text Display:** Upon selection, the answer appears in a dialogue-text box (as in battle) or as a subpanel. Then the next four questions overlay. This simulates Emerald’s move text. We should animate the text reveal (like Pokémon dialogue) for answers.
- **Styling:** Use the battle text box style for answers. Move-choice window inherits the panel style (light interior, dark border).
- **Cursor/Selection:** The move grid (topics) highlight on arrow and selection, identical to fight commands.
- **Presentation:** Keep each topic short phrase (as in examples). Answers are normal sentences.

This preserves Emerald’s familiar 2×2 menu interaction. For example, see how Emerald shows four moves under the command box with one highlighted by an arrow  – mimic that exactly for “DETAILS” depth menus.

Dialogue / Challenger UI

Emerald dialogue boxes (NPC conversations) appear as a text window at bottom with a little Pokéball icon (indicator) in the corner  . Important traits:

- **Dimensions:** Full-screen width (240px) and about 3–4 lines tall (~48px plus borders).
- **Text Appearance:** White text on black (dark) background? Actually GBA Emerald dialogue text is black on light gray, but the example suggests a grayish box with black text and a small arrow icon in bottom-right. The continuation arrow blinks when page is done.
- **Speaker:** There is no explicit name label. In Emerald, NPC dialogue is just text with no name. Tone is concise and uses quotation marks rarely.
- **Text Speed:** Letters appear with sound (if SFX on). We can mimic instant reveal or slow text. For authenticity, a moderate reveal speed (30–50ms per char) is best.
- **Interaction:** Player hits confirm (A) to speed up/finish.
- **Example lines:** Short, exclamatory sentences (often 1-2 clauses). The challenger’s lines should be casual/provocative, e.g. *“Interesting.”*, *“Tell me more about that.”* as given. They feel like one-liners, not paragraphs.

For our NPC (challenger), we will implement context-sensitive dialogue. E.g. after sending a project: *“That’s a nice project, huh?”* or one of the sample lines. The key is timing: text appears in the box and then the arrow blinks. The arrow icon (Pokéball) at the bottom right corner of the box indicates “more”  . Use the same white arrow with black outline.

We should provide 3–5 possible lines per audience type. Use them randomly or cyclically. Visually, the dialogue box should match Emerald’s (dark top highlight, light fill, layered border). The arrow icon (Pokéball) should be in the box corner.

Link Interaction

When the visitor selects **LINK**, we should prompt a confirmation in Emerald style. Pokémon typically asks “A link? Yes/No”. We can do similar: show a small dialog window with “Go to [DEST]?” or “Open [LINK]? (A)Yes (B)No”.

- **Window:** Use a smaller panel (not full-screen) with layered border. Perhaps center it or place it near bottom. The style is same layered frame with dark outline.
- **Content:** One line question (pixel font), and two options below (“Yes” and “No”) arranged horizontally. The selection arrow points at “Yes/No” options (like Yes/No in a generic game dialog).
- **Interaction:** If “Yes”, navigate to the link. If “No”, close dialog. (This is optional for speed; could skip and just open link). Pokémon menus often do not confirm leaving, but since it’s a website, safety is good.
- **Aesthetic:** Keep it quick. Possibly play a beep or show arrow blink. No fancy transition, just fade-in of window.

We will treat LINK like a battle choice, but with only “Yes”/“No”. Use similar fonts and arrow. Make sure the window’s color palette (perhaps black outline, light interior) does not clash with background.

Cursor & Focus States

Emerald uses a simple **arrow cursor** or Pokéball pointer to show selection (no mouse cursor since GBA). The arrow is a 8×8 white triangle with a black border, pointing right, as seen in `【40†embed_image】` and `【44†embed_image】`. We will replicate this arrow icon exactly. It moves in steps to the selected row or button.

For keyboard/mouse, emulate the arrow. For example, if the player uses keyboard or controller, highlight moves with the arrow. If using mouse/touch, we may highlight a row by a hover effect or click highlight (though original GBA has no hover).

Thus define:

- `cursorIcon`: a pixel arrow graphic (16px tall with shadow).
- Focus rectangle: Alternatively, when using keyboard, we can also highlight the entire row in a subtle way (like a slight background tint). But primarily use the arrow.
- When a button is inactive, reduce its opacity (not in Emerald, but good for UX).

All menu navigation should feel like using an old game. For example, pressing Down moves the arrow one row (snapping exactly 16px). The arrow can animate (blink or move quickly). Ensure the arrow never goes outside window borders.

No separate “hover” in classic sense; if implementing mouse, clicking an item should also show the arrow on that item. The focus styles must match Emerald’s crisp aesthetic (no CSS glows; use color changes or small arrow movements).

Iconography

Emerald’s icons are tiny pixel symbols. For example, the Bag and Pokéballs in `【44】`, type icons in Pokédex (like colored dots or small images). We need original icons for: Pokédex, Projects, Experience, Bag,

Trainer Card, Options, Resume, GitHub, LinkedIn, Email, project Types. These should be 16×16 or 32×32 pixel art. They should be simple silhouettes in 1–3 colors. For instance:

- **Pokédex:** A small rectangular “book” with a Pokéball or “P” on it.
- **Projects:** Maybe a folder or gear icon.
- **Experience:** A briefcase or building.
- **Bag:** A satchel or backpack.
- **Trainer Card:** An ID card or silhouette head.
- **Options:** A gear/cog.
- **Resume:** A document icon.
- **GitHub/LinkedIn/Email:** Simplified versions of their logos (e.g. “Octocat” shaped, “in” letters, envelope). Keep them monochrome or 2-color to fit pixel style.
- **Project Types:** Each type (Backend, AI, Blockchain, etc.) gets a small badge icon (like a microchip, brain, chain link).

All icons should use the same color style as Emerald (solid fills, dark outlines). For example, Bag icon can be brown with black outline; GitHub icon black silhouette. Type icons could use Emerald’s original type colors (e.g. red for Fire-type might translate to “Blockchain” in red, blue for “Frontend”). A consistent limited palette (use only the UI accent colors) will tie them together. Each should fit in a square grid exactly.

We have no direct sources, but the design approach is to mimic Emerald’s simplicity and clarity in icon design.

Animation

Emerald animations are simple but purposeful. We should identify a few *key animations* to mimic:

- **Sprite Walk Cycle:** 3–4 frames at ~6–8 FPS (45–60ms/frame). Looping when moving
【16tembed_image】.
- **Dialogue Text Reveal:** Characters appear one by one (~50ms per char) with a subtle tick sound. When complete, Pokéball arrow blinks at ~2Hz.
- **Menu Transitions:** Sliding windows. Emerald’s menus often slide/fade in quickly (~0.2–0.3s). For example, opening the main menu or bag often has a horizontal swipe effect. We can animate the panel dropping down from the top or fading in (duration ~0.25s).
- **Cursor Movement:** Instant or quick interpolation. The arrow can “jump” or slide to a new row in ~0.1s.
- **Battle Animations:** When a project (Pokémon) is sent out: the sprite might flash in or move from off-screen (Emerald does a quick pop-in). We can have the project icon or sprite slide up or fade in at start of battle.
- **Selection Feedback:** Pressing A on a menu option might briefly darken it or play a small flash.
- **Minimal Idle Motion:** The trainer’s sprite might have a slight bob or blink every few seconds (we can add a one-frame head tilt).

We list some animations and timing (approx):

Animation	Duration	Style	Purpose
Walk cycle (per step)	~0.3s total	3 frames at ~10 FPS	Movement in overworld

Animation	Duration	Style	Purpose
Dialogue text	~50ms/char	Typing effect, arrow blink	Simulate Emerald dialog
Menu open/close	0.2–0.3s	Slide or fade in/out	Smooth UI transitions
Cursor move	~0.1s	Immediate/jump or quick slide	Visual response to input
Selection flash	0.1s	Invert or darken frame	Confirm button press
Project entry	0.2s	Sprite fade/slide in	“Sent out [PROJECT]!” effect

These are **must-have** to capture feel (walk, dialog, menu transitions). Additional (nice-to-have) could be blinking lights, screen shake on selection, small bounce on buttons – but not critical. We should avoid heavy animations (keep pixel-perfect, no interpolation that blurs pixels).

Sound

While no copyrighted music, the *style* of Emerald audio cues can be emulated. Key sound cues:

- **Menu Cursor:** A short “tick” or “click” sound when moving cursor, like Emerald’s softer tone.
- **Confirm (A press):** Slightly higher beep than cursor.
- **Cancel (B press):** A different tone (Emerald uses a lower plop for cancel).
- **Text Scroll:** A faint typing blip per character (like Emerald’s text sound).
- **Battle Send-Out:** A brief flourish or whoosh when the project appears.
- **Battle Menu Select:** Clicking sound when selecting DETAILS/LINK, similar to Pokémon fight menu.
- **NPC Dialogue End:** Possibly Emerald has a low ‘pop’ when text is done or arrow appears. We can add a small sound with the blinking arrow.

All sounds should be short ($\leq 200\text{ms}$) and chiptune-like (8-bit synth). We do **not** use any actual Pokémon tunes or effects. Instead, use public-domain or original 8-bit style effects inspired by Emerald’s palette. The goal is subtle nostalgia, not a full remake soundtrack.

Pixel Rendering

To preserve the sharp pixel art on modern displays:

- **Nearest-Neighbor Scaling:** When scaling canvases or images, use CSS `image-rendering: pixelated;` (or equivalent) so pixels enlarge without blur.
- **Canvas Resolution:** Render at the internal 240×160 resolution (or a multiple thereof) and then scale up. Don’t allow fractional scaling. For example, display at 2× (480×320) or 3× (720×480).
- **Device Pixel Ratio:** Account for retina screens by doubling canvas resolution if needed. For WebGL or Canvas, set `imageSmoothingEnabled = false`.
- **Spritesheets:** Use integer scaling and keep animations on 1px grid.
- **Responsive:** On resizing, maintain aspect ratio. No stretching that breaks pixel ratio. Add letterboxing if needed (black bars).

For Phaser (if used): set `pixelArt: true` in game config to disable antialiasing. For CSS, use `background-size: contain` with `image-rendering: crisp-edges`.

These ensure our art stays crisp, matching Emerald's precise pixel look on any device.

Responsive Design

Emerald was fixed on 3:2 screens. For modern devices:

- **Internal Resolution:** Keep the 240×160 (3:2) ratio canvas. On larger screens, center it with a scaled pixel display. On wide monitors, show sidebars or repeat background tile art. On tall phones, letterbox or add UI around the game view.
- **Scaling:** On desktop, we can fit to window height (2× or more) and center horizontally. On mobile (portrait), we have less width: possibly letterbox top/bottom or rotate to landscape by default. One idea: force landscape orientation for game view, with on-screen controls overlaid in black bars.
- **Mobile Controls:** If game uses touch, overlay a semi-transparent GBA-style D-pad and A/B buttons (like old mobile ROM adapters). We should design stylized buttons matching Emerald color scheme (gray/green transparent). Map them so: Left side D-pad, right side A/B, plus two smaller X/Y at top corners. Ensure these have enough touch area (~48px or more). They should use the same pixelated border style (rounded squares with border) or at least low-res icons.
- **Layout Changes:** Dialogue boxes and menus could take more vertical space on small screens (since less horizontal px). Possibly wrap text to 2 lines. Or allow vertical scrolling of text.
- **Nested UI (Details):** The DETAILS Q&A might need a different layout on portrait (maybe single column list instead of 2×2 grid). But try to keep 2×2 and allow scrolling if needed.
- **UI Scaling:** Pixel fonts might look tiny on high-res phones; it's better to scale the entire canvas uniformly than to use larger font. So always scale the canvas up.

In summary, lock the game logic to 240×160, and on each device scale up by integer factor. Handle orientation explicitly and ensure on-screen controls appear only on touch devices (with an option to hide).

Design System Extraction

Based on all above analysis, we can extract reusable tokens and styles:

- **Canvas:** `width=240px, height=160px` (internal), aspect=3:2. Scaled by integer factors for display.
- **Grid:** Base unit = 8px. Tile = 16px. Spacing increments = 4px.
- **Typography:** Pixel font (monospace), body ~8px, title ~12px, line-height ~10px. All uppercase.
- **Borders:** `borderOuter = 2px solid (dark); borderInner = 1px solid (light accent); cornerRadius = 0px; padding = 4px`. Corner graphics use stepped squares.
- **Colors:**
 - `color.world.grass = #??` (bright green), `color.world.water = #??` (bright blue), `color.world.path = #??` (tan), etc.
 - `color.ui.borderOuter = darkGreen, borderInner = lightGreen; background = #fdf5c4 (cream); text = #000; select = #0ff (cyan); disabled = #888; warning = #f44; link = #29f (blue).`
- **Sprites:** Player/NPC sprite ~32×32, max 3 walk frames, 4 dir. Use 3–6 colors per sprite plus black outline.

- **Animation:** `anim.walkFrameTime ≈ 100ms`, `anim.menuTransition ≈ 250ms`, `anim.cursorMove ≈ 100ms`, `anim.dialogCharTime ≈ 50ms`.
- **Controls:** Keyboard arrows for menu, Enter/A = confirm, Escape/B = back, X = open menu, Y = open bag. Touch: on-screen D-pad + A/B/X/Y overlays.

These design tokens should be coded into CSS variables, Phaser configs, or a shared constants file.

Pokémon Emerald Visual DNA

From direct observation, Pokémon Emerald's "visual DNA" can be summarized as follows:

- **Strict Pixel Grid:** Everything aligns to a consistent 8px/16px grid, creating a tidy blocky look.
- **Limited Color Ramps:** Each element uses a small palette (2–4 shades plus outline), resulting in a harmonious but vibrant scene.
- **Thick Pixel Outlines:** Characters and many props have bold 1px outlines (often black) that make them pop.
- **Layered 2px Borders:** Windows use a two-tier border (dark then bright) that's reused across the UI.
- **Orthographic Overhead View:** Environments are drawn from a top-down angle with no perspective skew, giving a classic "flat" RPG feel.
- **High Density of Detail:** Even in a small resolution, Emerald packs many tiles and sprites on-screen (grass blades, flowers, etc.), giving life to the scene.
- **Pixel-Perfect Typography:** Text is small, monospaced pixel font, uniform and blocky. Every letter occupies its pixel grid square.
- **Simple Animations:** Movements and transitions are quick but use a few key frames for character walks or menu pops, reinforcing the pixel style.
- **Blue/Green UI Theme:** Predominantly green for borders/backgrounds (Emerald's namesake color), blue accents in battles. Warm cream/white fills for text areas.
- **Iconic Cursor:** A single, recognizable white arrow with black outline as cursor across all menus.
- **Grid-Based Composition:** Sprites, text, and HUD elements all follow a snapping grid, reinforcing order.

These rules are all evidenced by the actual game graphics (e.g. how [16] and [40] look) rather than generic retro concepts.

Original Portfolio Visual Language

DO (Emerald-inspired)	DON'T (Overdo Gamefication)
Use strict 16px grids for tiles and sprites.	DO NOT copy any Nintendo art or sprites.
Limit colors per element; use high-contrast outlines.	DON'T use realistic/painterly art – stick to pixel style.
Apply layered window borders (dark+light).	DON'T reproduce screens pixel-for-pixel.
Use nearest-neighbor scaling (pixelated look).	DON'T use full-screen video or smooth fonts.
Keep text short and incremental (like game dialog).	DON'T force cheesy game lingo.
Animate with 2–4 frame loops and quick UI transitions.	DON'T include long fade-outs or cinematic motion.

DO (Emerald-inspired)	DON'T (Overdo Gamefication)
Use original pixel icons for UI (resume, GitHub) in Emerald style.	DON'T just drop in copyrighted logos—redesign them pixel by pixel.

Keep what's Emerald-like: grid layout, border style, pixel fonts, limited palette, sprite outlines.

Reinterpret: Replace Pokémon with personal brand sprites and icons. E.g. a code icon instead of a Pokéball. Use palette philosophy (greens/blues) but tweak hues.

Modernize: Responsive layouts, accessible font rendering, online linking.

Create from scratch: All graphics (maps, character, NPCs) must be original. Use Emerald as *inspiration*, not a template.

The site should *feel* like Emerald (same UI grammar and art style) but never a direct copy. Visitors should think “this is like a Pokémon game UI” without any exact sprite reuse.

Screen-by-Screen Visual Blueprint

1. **Landing / Title:** A pixel-art title screen. Use a border similar to menu but with “Title” text. Possibly an animated logo (e.g. your name in Emerald font). Could show a simple overworld background. Transition with emerald’s fade-in “Nintendo” style.
2. **Overworld:** 240×160 game view with tiles (grass, path, buildings). Show the trainer sprite (you) on the map. The panel border does *not* apply here; this is game view. HUD: maybe a small “press Start” blinking text at bottom.
3. **Dialogue:** When talking to NPC, show Emerald-style text box at screen bottom (light gray fill, black text, white arrow). Whole screen dim below? Probably not. Maintain full brightness. Use [58] style arrow indicator.
4. **Player Menu:** Full-screen overlay with emerald-styled window (green border) `[40+embed_image]` . Left list (Projects, Experience, Bag, etc.), right info panel. Arrow cursor on selection.
5. **Bag Screen:** Overlaid window (as [44]) with top tabs (Documents, Profiles...) and two-pane list/detail. Cursor arrow on list items. Same border style.
6. **Pokédex List:** Window with a title bar. Left side shows scrollable list of skills (like party list), right side empty or icon grid. Highlight current.
7. **Pokédex Detail:** Window with layered border, top heading (skill name), icon (maybe top-left), and paragraph text. Possibly bottom “Type: X” badge. Use emerald font.
8. **Projects/Party:** Window (like `[54+embed_image]`) showing 6 project slots. Each slot: icon, name, type. Arrow on selected. Background likely tiled or plain.
9. **Battle:** Overlay: background art, two status panels (enemy top left, ally bottom right), trainer sprites on each side, Pokémon/project sprites center. A command box panel (DETAILS/LINK/PARTY/EXIT) on bottom right above a text box. Colors and layout exactly Emerald-like.
10. **Details Level 1:** When DETAILS chosen, show a new window (like fight menu) with 4 topics. 2×2 grid of buttons (like Pokémon moves) in layered border. Arrow on one. Top of window says maybe “Choose Topic”.
11. **Details Level 2:** Similar window with deeper topics. If none left, maybe display just the topic + answer in a dialogue box (with BACK option).
12. **Details Level 3:** One more depth (rare). Same style.
13. **Trainer Card:** Full-screen window styled like Emerald’s Trainer Card. Borders green/dark, background gold. Show avatar (left), name and condensed info (right). Pixel font for text. Possibly little icon decorations.
14. **Experience:** Window with a list of jobs, one per row (like a menu). Selected job’s details could appear in a subpanel at bottom or pop-up. Maintain emerald text boxes.

15. **Options:** Likely reuse Emerald's Option screen style (green border, black-blue menu background). Show settings (e.g. Sound: On/Off, Music: On/Off) in list form.
16. **Link Confirmation:** Small dialog window at center: question text and "Yes/No" options. Use emerald window style (e.g. black background with white text or vice versa) to confirm external link.
17. **Mobile Controller:** If on mobile, overlay semi-transparent GBA buttons (directional pad on left, A/B on right). Use pixel-art icons for D-pad and buttons in a pastel color (like translucent gray) with thin black border.
18. **Accessibility/Reduced Motion:** Optionally an "OPTIONS" screen allowing turn-off animations. If high-contrast needed, allow switching border/text to plain black/white. Always ensure text sizing is readable (e.g. double pixel font).

Each screen uses the color palette and typography specified. The composition (how elements are arranged) follows the Emerald examples. Cursors and animations as above. On mobile, controllers sit outside the game viewport, not covering text (i.e., appear in letterboxed areas or over background).

Prioritization

- **MUST (Match Emerald feel):** Pixel grid (16px tiles), sprite outlines, window border style, emerald-esque color accents (greens/blues), pixel font, arrow cursor, 3:2 aspect. These define the core aesthetic.
- **SHOULD:** Palette philosophy (limited ramps), walk animation, menu transitions, bag/pokedex layout imitation. Keeping the UI grammar (layers of border, consistent typography) is high priority.
- **CAN Be Modernized:** Some padding/margins (modern UI might use a bit more space), responsiveness (mobile UI), using the browser's capabilities.
- **CAN Be Original:** All actual artwork (characters, icons, backgrounds) must be original. Music/sound cues reimagined (no Nintendo sound). Content text (even if inspired by game) must be customized to portfolio.

In essence, we *must* duplicate the *style rules* of Emerald, but the *content* must be unique to the portfolio. Gameplay elements (battles, menus) will exist only as a metaphor, not actual game logic, so there's freedom in how deep those go. Focus effort on those visuals (borders, colors, fonts) that immediately signal "Emerald."

Final Implementation Blueprint

A. Canvas: Use an HTML5 canvas of **240×160px**. Set `image-rendering: pixelated`. Scale by integer factors. Lock to landscape 3:2.

B. Tile Spec: Base tile **16×16px** on game grid. World map built from 15×10 tile maps.

C. Sprites: Overworld player/NPC: **32×32px**, 4 directions, 3 frames walk. Project icons: 32×32. NPCs ~32×32. Use PNG spritesheets with nearest-neighbor scaling.

D. UI Components:

- **Window:** Outer border 2px, inner border 1px (RGB e.g. #000 and #3cc). Content background #fdf5c4 (cream). Corners square.
- **Dialogue Box:** Fill #cccccc, text #000. Pokéball arrow icon at bottom right.

- **Buttons:** In menu grids, use white text on dark-blue background for unselected, and invert for selected (or arrow indicator). Arrow = white triangle with black border (8px).
- **Indicators:** Use the same arrow graphic for selection cursors. A flashing Pokéball icon for “press more” in dialogue.

E. Typography: 1x pixel font (like Commodore 64 style) ~8px. Use uppercase. Line height ~10px. Monospace layout. For headings (screen titles), use bold or 10px variant. Ensure font scaling with canvas.

F. Colors:

- World: Grass #4CAF50, Dirt #B87030, Water #337AB7 (for example).
- UI Borders: Outer #004000 (dark green), Inner #00FF00 (bright).
- UI BG: #FDF5C4.
- Text: #000000.
- Selection: #00FFFF (cyan).
- Disabled: #AAAAAA.
- Warning: #F44336 (red).
- Link: #337AB7 (blue).

(These hex are illustrative; tweak to best visual match Emerald’s tone.)

G. Window/Panel Spec: All windows follow the border/padding rule above. Cursor must snap to text baselines. Use grid of 16px for layout inside windows.

H. Cursor: Create a sprite/icon 8×8 (with 2px stroke, 4px fill) of an arrow. On focus, place at left of selected item (offset properly, e.g. 8px from border). Animate blinking by toggling visibility at 0.5s rate.

I. Animation:

- Configure Phaser (or chosen engine) with fixed 60fps, but sprite update at ~6fps (every 0.15s).
- Menu open: tween window y from -windowHeight to 0 over 250ms.
- Dialogue text: reveal each char with 50ms timer. After full line, start arrow blink.
- Walk: cycle frames every ~150ms.

Use CSS/JS for UI transitions if not using canvas (like fadeIn for images).

J. Responsive:

- Detect screen size: calculate max integer scale to fit either height or width.
- If mobile (touch): show on-screen buttons (divs or canvas overlays) representing D-pad and A/B/X/Y. Style them in pixel-art frames (e.g. an 8px thick border square with transparent fill).
- For portrait: possibly letterbox on sides and move UI elements into black bars. E.g. list menus could go full screen, or prompt to rotate device.

K. Framework: Phaser recommended with `pixelArt: true`. Use tilemap layers for overworld. For UI, use Phaser Scenes for each screen with image and text objects (set font via bitmap font or CSS). Disable antialias.

L. CSS: For HTML fallback or logo, use `font-family` of a monospaced pixel font. For images, use `image-rendering: crisp-edges`. Use fixed-size containers.

M. Asset List:

- Tiles: grass, path, water, ground, building tiles (in original style).
- Sprites: trainer (4 dir), NPCs (variety), challenger (special sprite).
- UI frames: corner/edge sprites for windows (or draw via shapes).
- Icons: Bag, Pokedex, etc., type badges, on-screen buttons.
- Buttons: Arrow cursor, Pokéball.
- Sound: select, confirm, text char (create or free asset).

N. Folder Structure:

```
/assets
  /sprites
    player.png (32x32, 4 dir)
    npc1.png, npc2.png
    icons.png (bag, pokedex, github, etc.)
  /tiles
    tileset.png (16x16 tiles)
  /ui
    window_border.png (manual or generated)
    arrows.png
  /fonts
    pixelFont.fnt/.png (if using bitmap font)
  /audio
    select.wav, confirm.wav, text.wav
```

O. Design Tokens (code): In a `theme.js` or CSS:

```
--tile-size: 16px;
--font-size: 8px;
--color-border-outer: #004000;
--color-border-inner: #00FF00;
--color-bg: #FDF5C4;
--color-text: #000000;
--color-select: #00FFFF;
--animation-speed: 0.25s;
```

P. Component Hierarchy:

- `GameScreen` (overworld)
 - `MenuScreen` (Player Menu)
 - `BagScreen`, `PokedexList`, `PokedexDetail`, `PartyScreen`, `TrainerCard`, `ExperienceScreen`
 - `BattleScreen` (with subcomponents: `StatusPanel`, `CommandBox`, `DialogueBox`)
 - `DetailsScreen` (nested Q&A)
 - `LinkDialog`, `ConfirmDialog` (overlay screens)
- Each uses a base `Window` component for border. Cursor is shared global overlay.

Each screen reads from the same design system (colors, fonts). UI elements are composed from the tokens above. For example, `Window` component draws a rectangle with the two borders and background using those token colors, and positions child content inside with padding. Menus use a `MenuList` component that automatically spaces rows at 16px intervals and places an arrow for the selected index.

All these ensure consistency: e.g. changing `--color-border-outer` updates every border at once.

This design blueprint, guided by actual Emerald visuals 【69†L222-L224】 【65†L139-L142】 and references (embedded screenshots), should allow developers/designers to implement a complete Emerald-inspired UI. Each specification ties back to an observed Emerald characteristic.
